COMP 3920 Database Systems 1

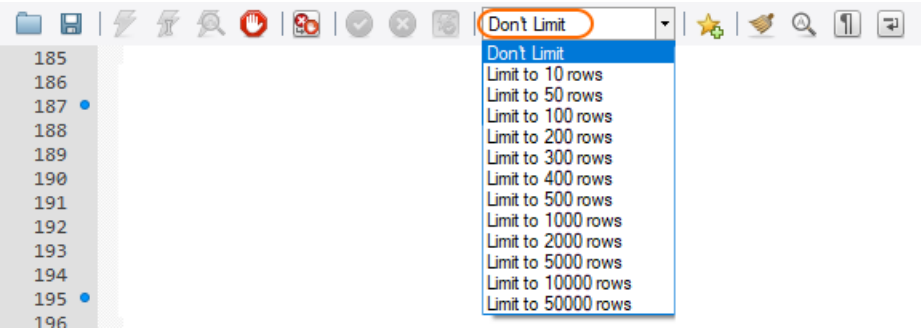
MySQL
Indexing

Introduction:
The goal of this document is to serve as a walkthrough working indexes on a large table, in MySQL 8.

Please complete "Slow Inserts" prior to starting this lab.

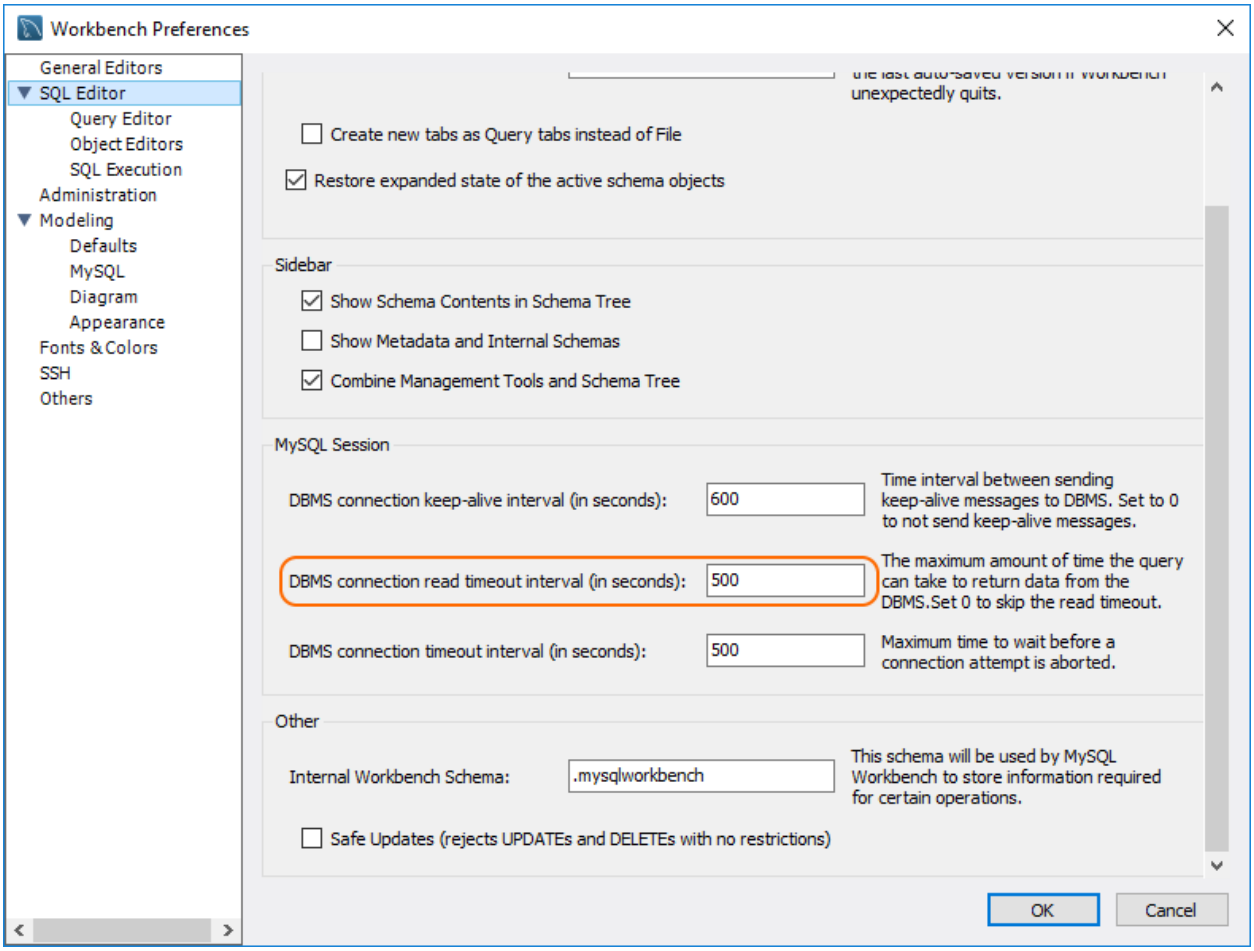
Using the data from the user table from inside the `lotsofdata` database we will be examining how indexes can help improve query performance and how a badly created (or partial) index will in little help to speed up slow queries.

Step 1:
If you are using MySQL Workbench, you will want to disable the limit by feature for your queries. The limit by will change the way MySQL evaluates the query and can cause the queries to be much slower.



Also, when running slow queries, MySQL workbench will timeout waiting for queries to finish. The default is 30 seconds. Increase this timeout by going to Edit → Preferences. You can increase "read timeout interval" to 600 (600s = 10 minutes).

IMPORTANT: Close and re-open MySQL Workbench for the changes to the settings to take effect.

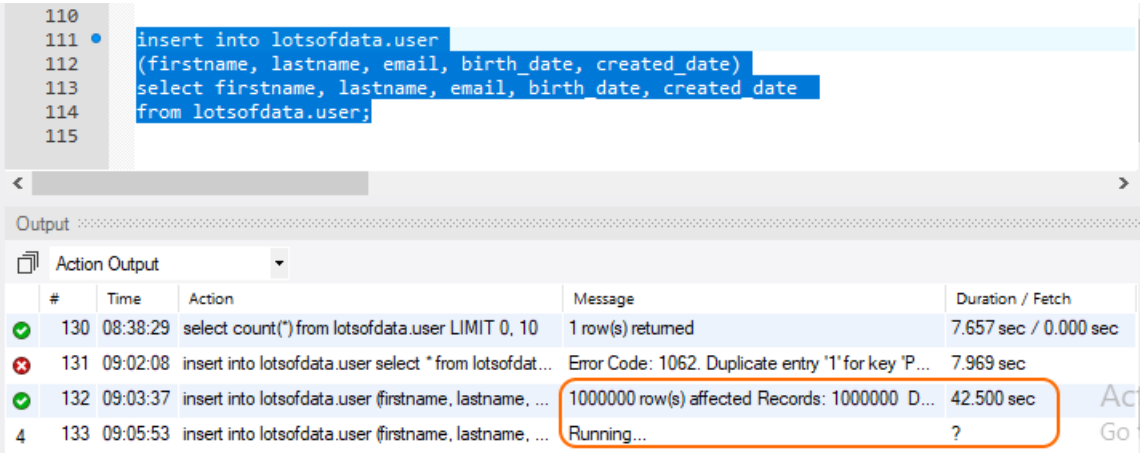

Workbench Preferences
General Editors
SQL Editor
Query Editor
Object Editors
SQL Execution
Administration
Modeling
Defaults
MySQL
Diagram
Appearance
Fonts & Colors
SSH
Others
unexpectedly quits.
Create new tabs as Query tabs instead of File
Restore expanded state of the active schema objects
Sidebar
Show Schema Contents in Schema Tree
Show Metadata and Internal Schemas
Combine Management Tools and Schema Tree
MySQL Session
DBMS connection keep-alive interval (in seconds):
600
Time interval between sending keep-alive messages to DBMS. Set to 0 to not send keep-alive messages.
DBMS connection read timeout interval (in seconds):
500
The maximum amount of time the query can take to return data from the DBMS.Set 0 to skip the read timeout.
DBMS connection timeout interval (in seconds):
500
Maximum time to wait before a connection attempt is aborted.
Other
Internal Workbench Schema:
.mysqlworkbench
This schema will be used by MySQL Workbench to store information required for certain operations.
Safe Updates (rejects UPDATEs and DELETEs with no restrictions)
OK
Cancel

Step 2:
We should now have 1,000,000 rows in our `lotsofdata.user` table. But I'm gonna be greedy and I want MORE! Let's artificially increase the size of our table by duplicating the number of rows a few times.

We will run this command to duplicate the rows in our table:

```
INSERT INTO lotsofdata.user
(firstname, lastname, email, birth_date, created_date)
SELECT firstname, lastname, email, birth_date, created_date
FROM lotsofdata.user;
```

Run this query **3 times**, until we have at least 8,000,000 (8 million) rows in our table.
These queries will take some time; you are working with several million rows at a time now! Also, the more you run this query, the more rows are being added so the query will get slower each time you run it.



If you run into this error:
The total number of locks exceeds the lock table size.

Output

Action Output

#	Time	Action	Message	Duration / Fetch
130	08:38:29	select count(*) from lots...	1 row(s) returned	7.657 sec / 0.000 sec
131	09:02:08	insert into lotsofdata.use...	Error Code: 1062. Duplicate entry '1' for key 'PRIMARY'	7.969 sec
132	09:03:37	insert into lotsofdata.use...	1000000 row(s) affected Records: 1000000 Duplicates: 0 Warnings: 0	42.500 sec
133	09:05:53	insert into lotsofdata.use...	Error Code: 1206. The total number of locks exceeds the lock table size	211.000 sec

Follow these instructions to increase the InnoDB lock size.

Go to my.ini (Windows).

In Windows you'll find the parameter:
```
innodb_buffer_pool_size=8M
```

Increase this value to 512M like so.

```
innodb_buffer_pool_size=512M
```

Save my.ini and restart MySQL.

We need to confirm we have 8 million rows in our table.
Run a count of the rows on `lotsofdata.user`:

Submit your screenshot – include the execution time:

07_eight_million_rows.jpg

Submit

```
mysql> select count(*) from lotsofdata.user;
+----------+
| count(*) |
+----------+
|          |
+----------+
1 row in set (      sec)
```

Let's run another query:

```
SELECT min(birth_date), max(birth_date) FROM lotsofdata.user;
```

Submit your screenshot – include the execution time:

08_oldest_to_youngest_slow.jpg

Submit

```
mysql> select min(birth_date), max(birth_date) from lotsofdata.user;
+---------------------+---------------------+
| min(birth_date)     | max(birth_date)     |
+---------------------+---------------------+
|                     |                     |
+---------------------+---------------------+
1 row in set (      sec)
```

These queries are SLOOOOOOW…
Imagine one of these queries needs to be run in order to show information on a website. Would you wait almost a minute for a website to load? Most people get impatient after 1 second of waiting!

We can do better.

Let's put an index on `birth_date`.

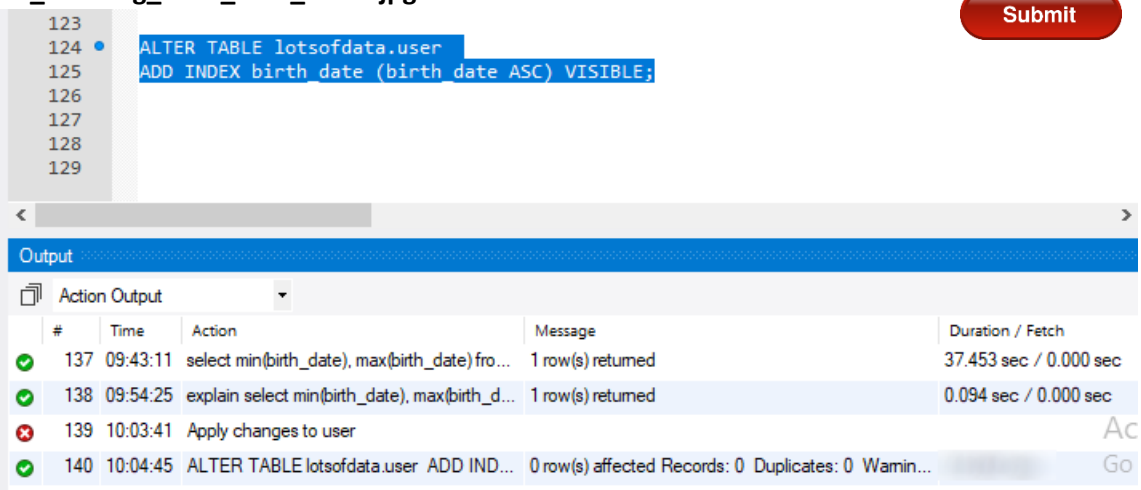
```
ALTER TABLE lotsofdata.user
ADD INDEX birth_date (birth_date ASC) VISIBLE;
```

We'll need to give it some time to create the index. There are a lot of rows and this is not a fast process.

Submit a screenshot of the execution time of creating the index.

09_Creating_birth_date_index.jpg

Submit

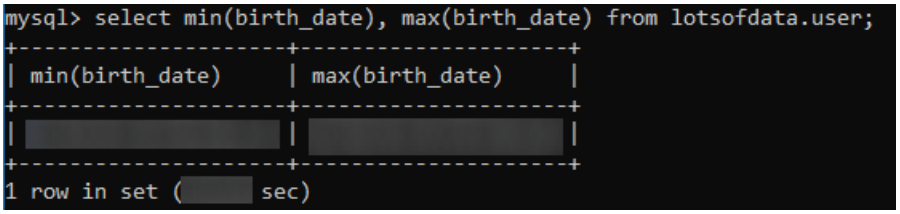


It's a lot of work upfront to create the index (not for us, but for MySQL) but now let's run our oldest to youngest query again:

Submit your screenshot – include the execution time:

10_oldest_to_youngest_fast.jpg

Submit



A bit better? I think so.

Let's use the EXPLAIN command to show the difference between the execution of the unindexed and indexed oldest to youngest queries.

The EXPLAIN command can be added in front of many queries in MySQL to see a summary of how MySQL looks at your query and how will attempt to provide you the results. It can help us to optimize poorly performing queries by realizing we might not be taking advantage of the optimizations we could be.

A cool new feature of MySQL is the ability to make an index invisible. Making an index invisible will prevent MySQL from seeing it while it tries to calculate the best way to execute a query. This give us the ability to temporarily disable an index to see how the performance of our queries does with and without the index. Note that the index is still there, (i.e. we don't need to spend a lot of time recreating the index when we want to re-enable it) we can make it visible again at any time.

To make the birth_date index invisible we use this command:

```
ALTER TABLE lotsofdata.user ALTER INDEX birth_date INVISIBLE;
```

Make the `birth_date` index invisible and let's run our oldest to youngest query again to prove it's slow again.

```
mysql> select min(birth_date), max(birth_date) from lotsofdata.user;
+---------------------+---------------------+
| min(birth_date)     | max(birth_date)     |
+---------------------+---------------------+
| [redacted]          | [redacted]          |
+---------------------+---------------------+
1 row in set ([redacted] sec)
```

Then let's use the EXPLAIN command to show why it is slow:
Add EXPLAIN to the front of our oldest to youngest query and run it.

Submit a screenshot of your results:

11_explain_oldest_to_youngest_slow.jpg

Submit

```
mysql> explain select min(birth_date), max(birth_date) from lotsofdata.user;
+----+-------------+-------+------------+------+---------------+------+---------+------+---------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows    | filtered | Extra |
+----+-------------+-------+------------+------+---------------+------+---------+------+---------+----------+-------+
|  1 | SIMPLE      | [redacted]                                                                                          |
+----+-------------+-------+------------+------+---------------+------+---------+------+---------+----------+-------+
1 row in set, 1 warning (0.02 sec)
```

Re-enable the index by making the index visible again:

```
ALTER TABLE lotsofdata.user ALTER INDEX birth_date VISIBLE;
```

Submit a screenshot of your results:

12_explain_oldest_to_youngest_fast.jpg

Submit

```
mysql> explain select min(birth_date), max(birth_date) from lotsofdata.user;
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra                   |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------------------+
|  1 | SIMPLE      | NULL  | [redacted]                                                                                                                |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------------------+
1 row in set, 1 warning (0.00 sec)
```

What is the difference in the EXPLAIN command for the oldest and youngest query when run with the index visible and invisible?

What rank on the Explain Command Scale is the oldest and youngest slow query?

What rank on the Explain Command Scale is the oldest and youngest fast query?

Step 3:

Run the following queries and rank them on the Explain Command Scale:

Query #1:

```
SELECT lastname, firstname, birth_date, email
FROM lotsofdata.user
WHERE user_id = 1;
```

Capture a screenshot of the execution time and the explain command:

13_query_one.jpg

Submit

```
mysql> select lastname, firstname, birth_date, email from lotsofdata.user where user_id = 1;
+----------+-----------+---------------------+---------------------+
| lastname | firstname | birth_date          | email               |
+----------+-----------+---------------------+---------------------+
| Long     | Zahir     |[redacted]|
+----------+-----------+---------------------+---------------------+
1 row in set ([redacted])

mysql> explain select lastname, firstname, birth_date, email from lotsofdata.user where user_id = 1;
+----+-------------+-------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type  | possible_keys | key     | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | user  |[redacted]|
+----+-------------+-------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.01 sec)
```

What rank on the Explain Command Scale is Query #1?

Query #2:

```
SELECT lastname, firstname, birth_date, email
FROM lotsofdata.user
WHERE birth_date = '1943-04-12 09:33:02';
```

Capture a screenshot of the execution time and the explain command:

Submit

14_query_two.jpg

```
mysql> select lastname, firstname, birth_date, email from lotsofdata.user where birth_date = '1943-04-12 09:33:02';
+----------+----------+---------------------+-----------------------------------------+
| lastname | firstname | birth_date          | email                                   |
+----------+----------+---------------------+-----------------------------------------+
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
| Welch    | Timothy  |                     |                                         |
+----------+----------+---------------------+-----------------------------------------+
8 rows in set (      )

mysql> explain select lastname, firstname, birth_date, email from lotsofdata.user where birth_date = '1943-04-12 09:33:02';
+----+-------------+-------+------------+------+---------------+------------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key        | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | user  |                                                                                                |
+----+-------------+-------+------------+------+---------------+------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)
```

What rank on the Explain Command Scale is Query #2?

Step 4:

<u>The importance of good indexes.</u>

Simply adding an index to a table won't speed up the execution time of all your queries. You need to add the *correct* indexes. Adding the wrong index will only increase your table size and fill your storage space with useless information if the index is never used. Keep in mind that in some cases indexes can take up as much space as the actual table itself! Also keep in mind that sometimes, even with the best indexes, indexes aren't enough to solve the problem of slow queries. We sometimes need other techniques to get the performance we need.

Our next query, Query #3 is:

```
SELECT count(*), lastname
FROM lotsofdata.user
WHERE lastname LIKE "A%"
AND birth_date >= "1977-01-01 0:00:00"
GROUP BY lastname;
```

Before we run the query, let's see what the EXPLAIN command has to say.

What rank on the Explain Command Scale is Query #3?

Submit a screenshot of the Explain Command (use the \G option – instead of a semicolon at the end to make it fit on the screen):

```
SELECT count(*), lastname
FROM lotsofdata.user
WHERE lastname LIKE "A%"
AND birth_date >= "1977-01-01 0:00:00"
GROUP BY lastname\G
```

15_explain_query_three_birth_date_index.jpg

Submit

```
mysql> explain select count(*), lastname from lotsofdata.user where lastname like "A%" AND birth_date >= "1977-01-01 0
:00:00" group by lastname \G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: user
   partitions: [redacted]
         type: [redacted]
possible_keys: [redacted]
          key: [redacted]
      key_len: [redacted]
          ref: [redacted]
         rows: [redacted]
     filtered: [redacted]
        Extra: [redacted]
1 row in set, 1 warning (0.04 sec)
```

Does our Query #3 use our `birth_date` index?

<u>Query #3 – Run #1:</u>

Let's run the query, but let's also be ready to stop the query if it takes too long to run.
Let the query run for 3 minutes (180 seconds), if it doesn't complete in time you can abort it.
To see how long a query has run, and to stop a query that is taking a long time we can open another command line, login to mysql as root and run the following commands:

```
SHOW FULL PROCESSLIST \G
```

Find your query in the list and if Time is > 180 seconds you can abort the query.

```
*************************** 5. row ***************************
     Id: 33
   User: root
   Host: localhost:60239
     db: testing
Command: Query
   Time: 15
  State: Sending data
   Info: select count(*), lastname from lotsofdata.user where lastname like "A%" AND birth_date >= "1977-01-0
```

Find the ID of the process that matches your query and execute the following query (replace <ID> with your ID – in my example ID = 33):

```
KILL <ID>;
```

Next create an index for `lastname` and `birth_date` as a multi column index:

```
ALTER TABLE lotsofdata.user
ADD INDEX lastname_birth_date (lastname ASC, birth_date ASC) VISIBLE;
```

Disable the `birth_date` index by making it invisible:

```
ALTER TABLE lotsofdata.user ALTER INDEX birth_date INVISIBLE;
```

16_explain_query_three_lastname_birth_date_index.jpg

Submit

```
mysql> explain select count(*), lastname from lotsofdata.user where lastname like "A%" AND birth_date >= "1977-01-01 0
:00:00" group by lastname \G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: user
   partitions: [redacted]
         type: [redacted]
possible_keys: [redacted]
          key: [redacted]
      key_len: [redacted]
          ref: [redacted]
         rows: [redacted]
     filtered: [redacted]
        Extra: [redacted]
1 row in set, 1 warning (0.04 sec)
```

Does our Query #3 use our `lastname_birth_date` index?

<u>Query #3 – Run #2:</u>

Run Query #3 again.

```
|      3072 | Alford    |
|      3088 | Allen     |
|      2920 | Allison   |
|      3096 | Alston    |
|      3208 | Alvarado  |
|      3080 | Alvarez   |
|      3104 | Anderson  |
|      2672 | Andrews   |
|      3024 | Anthony   |
|      3048 | Armstrong |
|      2904 | Arnold    |
|      3184 | Ashley    |
|      3240 | Atkins    |
|      3144 | Atkinson  |
|      3288 | Austin    |
|      2960 | Avery     |
|      2976 | Avila     |
|      2920 | Ayala     |
|      2936 | Ayers     |
+----------+-----------+
28 rows in set ([redacted])
```

Does our Query #3 use our `lastname_birth_date` index?

Which Run for Query #3 was faster? Run #1 or Run #2?

Next create an index for `birth_date` and `lastname` as a multi column index (in the opposite order):

```
ALTER TABLE lotsofdata.user
ADD INDEX birth_date_lastname (birth_date ASC, lastname ASC) VISIBLE;
```

Disable the `lastname_birth_date` index by making it invisible:

```
ALTER TABLE lotsofdata.user ALTER INDEX lastname_birth_date INVISIBLE;
```

17_explain_query_three_birth_date_lastname_index.jpg

Submit

```
mysql> explain select count(*), lastname from lotsofdata.user where lastname like "A%" AND birth_date >= "1977-01-01 0
:00:00" group by lastname \G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: user
   partitions: [redacted]
         type: [redacted]
possible_keys: [redacted]
          key: [redacted]
      key_len: [redacted]
          ref: [redacted]
         rows: [redacted]
     filtered: [redacted]
        Extra: [redacted]
1 row in set, 1 warning (0.04 sec)
```

Does our Query #3 use our `birth_date_lastname` index?

Query #3 – Run #3:

Run Query #3 again.

```
|      3096 | Alexander |
|      3288 | Austin    |
|      2936 | Ayers     |
|      3336 | Acevedo   |
|      2904 | Aguirre   |
|      3272 | Adkins    |
|      3096 | Alston    |
|      3024 | Anthony   |
|      3144 | Atkinson  |
|      3296 | Albert    |
|      3240 | Atkins    |
|      3088 | Allen     |
|      3104 | Aguilar   |
|      3184 | Ashley    |
|      3072 | Alford    |
|      2976 | Avila     |
|      2920 | Allison   |
|      3000 | Abbott    |
|      2672 | Andrews   |
+----------+-----------+
28 rows in set (                    )
```

Which Run for Query #3 was faster? Run #2 or Run #3?

Why is `lastname_birth_date` index better than `birth_date_lastname` for Query #3?